Unit-3 - Syntax Analysis.

Syntax Analyzer takes input as tokens generated by lexical Analyzer, checks tokens are valid or not by passing tokens into the grammar. If grammar accepts the tokens, then it considers that tokens are valid and generate parse tree else generates Syntax error.

Role of Syntax Analyzer: (Role of Parser):

In the compiler model, normally parser obtains a string of tokens from the lexical analyzer and verifies tokens are valid or not. Below diagram depicts the position of Parser in the compiler model.

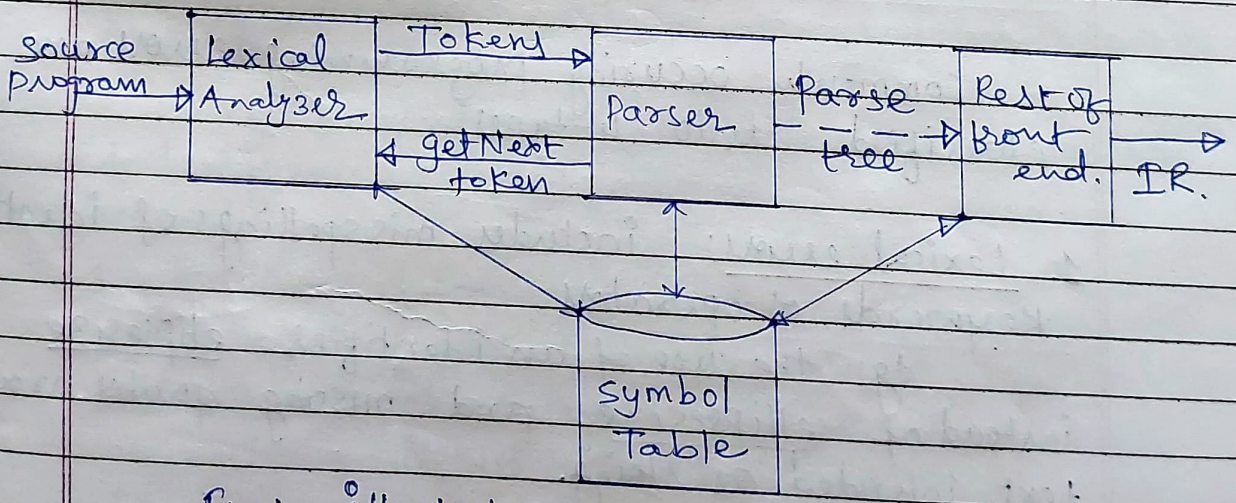


Fig:- illustrates position of Parser in the compiler model.

If tokens are accepted in the grammar, syntax Analyzer/parser generates parse tree.

To construct the parse tree, we have mainly two types of Parsing technique.

1. Top down Parsing technique
2. Bottom-up parsing technique

In top-down parsing technique, parse tree is constructed from top (root) to bottom (leaves), while in bottom up parsing technique, parse tree is constructed from bottom (leaves) to top (root).

VVIMP

Types of Errors and Error handling Strategies

If a compiler had to process only correct programs, its design and implementation would be simplified greately. However, a compiler is expected to assist the programmer in locating and tracking the errors that occurs despite the programmer's efforts.

Commonly occuring programming errors are classified into 4-types.

1. Lexical errors: includes misspellings of identifiers, keywords, or operators.
   Eg: the use of an identifier elipsesize instead of ellipsesize and missing quotes around text intended as string.

2. Syntactic errors: include misplaced semicolons or extra or missing braces that is "{" or "}".
   As another example, in C or Java, the appearance of a case statement without an enclosing switch is syntactic error.

3. Semantic error: include type mismatches between

operators and operands.

4. Logical error: can be anything from incorrect reasonning on the part of the programmer.

For eg: In C program writing assignment operator '=' instead of the comparison operator ==. Though '=' operator is correct but with respect to that particular program it is incorrect.

Error-Recovery Strategies:

Once an error is detected, the simplest approach is for the parser to quit with an informative error message. Some of the additional errors are often unnoticed, if the parser can correct itself and continue with further process.

Some of the automatic error recovery techniques are as follows.

1. Panic mode Recovery: In this error recovery technique, upon discovering error, the parser discards input symbols one at a time until remaining input matches any of the pattern so that tokens are generated. How input symbols are discarded is based on compiler developer on what logic he/she has used.

2. Phrase-Level Recovery: On discovering error, a parser may perform local correction on the remaining input; that is, it may replace a prefix of the remaining input by some string

that allows the parser to continue. A typical local correction is to replace a comma by semicolon, delete an extra semicolon, or insert a missing semicolon.

3. Error production: In beginning compiler developer need to identify all the possible errors, For these errors need to write a wrong grammar or production and need to incorporate into the language. Whenever input is accepted by these wrong productions of grammar, parser identifies these as an error and call any of the suitable error recovery technique to overcome the error.

4. Global correction: Like in the error production recovery technique, here also in the beginning compiler developer needs to identify all the possible errors. For these errors, we construct parse tree and store it in the computer memory. If input string is accepted by any of these parse tree, it considers there is error in the input and call any of the suitable error recovery technique to overcome the error.

Context Free Grammar:

Context free Grammar consists of
(i) terminal symbols
(ii) non-terminal symbols
(iii) Start symbol
(iv) Productions.

Page No.
Date

1. Terminal symbols: are the basic symbols from which strings are formed. Terminal symbols are normally considered as token name. Terminal symbols are last element in the language meaning that terminal symbols cannot be replaced by any other terminal or non-terminal symbols.

Some of the possible terminal symbols are:

(i) All lowel case letters
(ii) All punctuation symbols
(iii) All arithmatic and logical operators
(iv) All digits
(v) and anything written in boldface letter is also considered as terminal symbols.

2. Non-terminal symbols:→ Non-terminal symbols are normally represented by capital letters. Non-terminal symbols can be further replaced by any other terminal or non-terminal symbols.

3. In a grammar, one non-terminal is distinguished as the start symbol. Normally left side non-terminal symbol of first production is considered as start symbol.

4. Production: Production is considered as rewriting rule, is a rule of a grammar. Syntax of the production is

NT → Strings of terminal and non-terminal

left side of the production always contains single non-terminal and right side of the production contains either single terminal or non-terminal or combination of both terminal and non-terminal.

Page No.
Date

Eg: $E \rightarrow E+T/T$
$T \rightarrow T*f/f$
$F \rightarrow (E)/id.$

Above grammar is example for asithmatic operation + and *.

Here E, T & F are considered as non-terminal symbols.

+, *, (,) & id are considered as terminal symbols.

'E' start symbol of the grammar and,

$E \rightarrow E+T$
$E \rightarrow T$
$T \rightarrow T*F$
$T \rightarrow F$
$F \rightarrow (E)$
$F \rightarrow id$

are productions.

Ambigous grammar:

A grammar is said to be ambigous grammar, when it produces more than one parse tree for given input string.

How to eliminate ambiguity of the grammar?

Ambiguity of the grammar can be eliminated by rewriting / modifying the grammar.

for Eg:

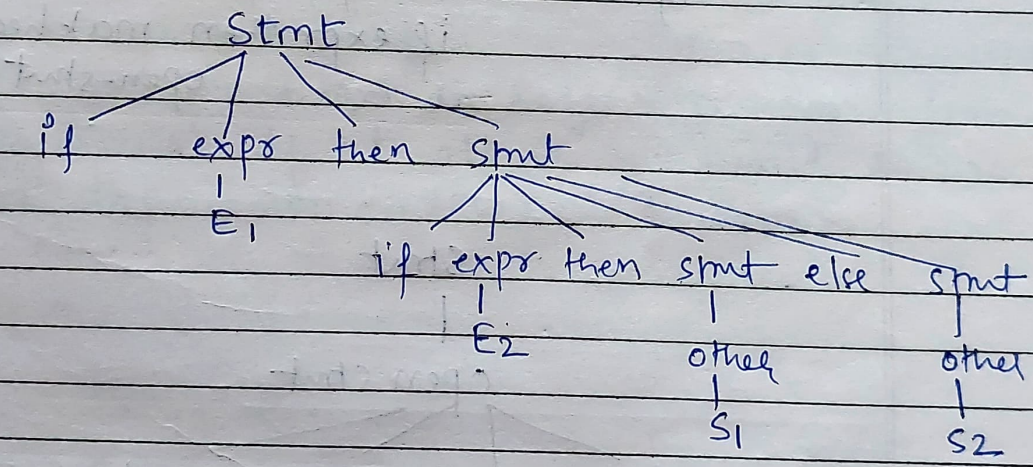
Let us consider a grammar

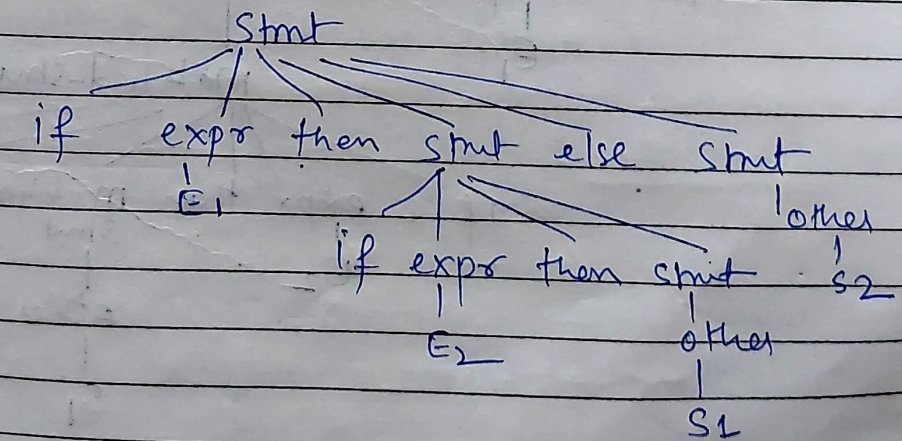
stmt → if expr then stmt | if expr then stmt else stmt | other

and input string is

if E_1 then if E_2 then S_1 else S_2

let us check how many Parse above grammar produces for the given Input string.



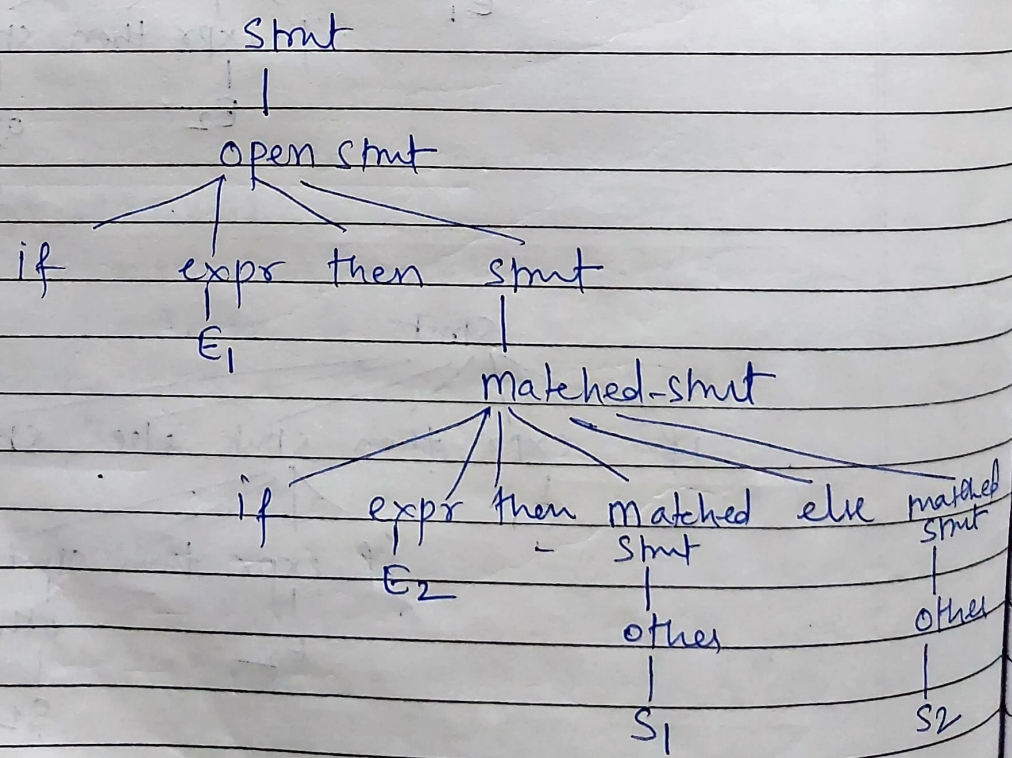


above grammar produces more than one parse tree for given Input string so, grammar is ambigous.

By rewriting the grammar, we can eliminate ambiguity of the grammar.
rewriting the grammar is completely depends upon compiler developer, what logic they have used based on that grammar is modified.

Let us modify the grammar and will check how many parse tree it generates.

stmt → matched_stmt | open_stmt
matched-stmt → if expr then matched_stmt
else matched-stmt | others
open stmt → if expr then stmt |
if expr then matched-stmt else
open-stmt



So, above modified grammar paeduces only one parse-tree hence, grammar is unambigous grammar.

Page No.
Date

Left Recursion grammar:-

A grammar is left recursive if it has a non-terminal A such that there is a derivation $A \rightarrow A\alpha$ for some string α.

Or

If Left side non-terminal symbol is equals to or same as that of left most non-terminal symbol of RHS side than the grammar is said to be Left recursive grammar.

Note:- * Top-down parsing technique does not supports grammar, if it contains left recursion in it. So, we need to left recursion from the grammar if at all if we want to use it for top-down parsing technique.

if the production is

$$A \rightarrow A\alpha | \beta$$

then left recursion can be eliminated by rewriting the above grammar as

$$A \rightarrow \beta A'$$
$$A' \rightarrow \alpha A' | \varepsilon$$

} Free from left recursion.

For Eg:-

Let us consider a grammar

$E \rightarrow E+T | T$

$T \rightarrow T*F | F$

$F \rightarrow (E) | id$

above grammar is left recursion since, first production, where left side Non-terminal symbol is equals to left most non-terminal symbol of RHS side. So left recursion can be eliminated using above rule as.

$E \rightarrow TE'$
$E' \rightarrow +TE' | \epsilon$
$T \rightarrow FT'$
$T' \rightarrow *FT' | \epsilon$
$f \rightarrow (E) | id.$

Algorithm for eliminating left recursion:

Input: Grammar G with no cycles or ϵ-productio

output: An equivalent grammar with no left recursion.

Method: Apply below algo. to eliminate left recursion.

1. arrange the non-terminals in some order $A_1, A_2, \ldots A_n$.
2. for (each i from 1 to n) {
3. for (each j from 1 to i-1) {
4. replace each production of the form $A_i \rightarrow A_j\gamma$ by the productions $A_i \rightarrow \delta_1\gamma | \delta_2\gamma | \ldots | \delta_k\gamma$, where $A_j \rightarrow \delta_1 | \delta_2 | \ldots | \delta_k$ are all current A_j productions.

}

}

Page No.
Date

Left Factoring :-

Left Factoring is a grammar transformation that is useful for producing grammar suitable for predictive or top-down parsing technique.

When the choice between two alternative A-productions is not clear, we may rewrite the productions in such way that there is no confusion in choosing A-production.

For Eg. if we have the production in general as

$$A \to \alpha\beta_1 | \alpha\beta_2$$

are two A-productions, and the input begins with a nonempty string derived from α, we do not know whether to expand A to $\alpha\beta_1$ or $\alpha\beta_2$. So, to overcome this problem we rewrite the above grammar as

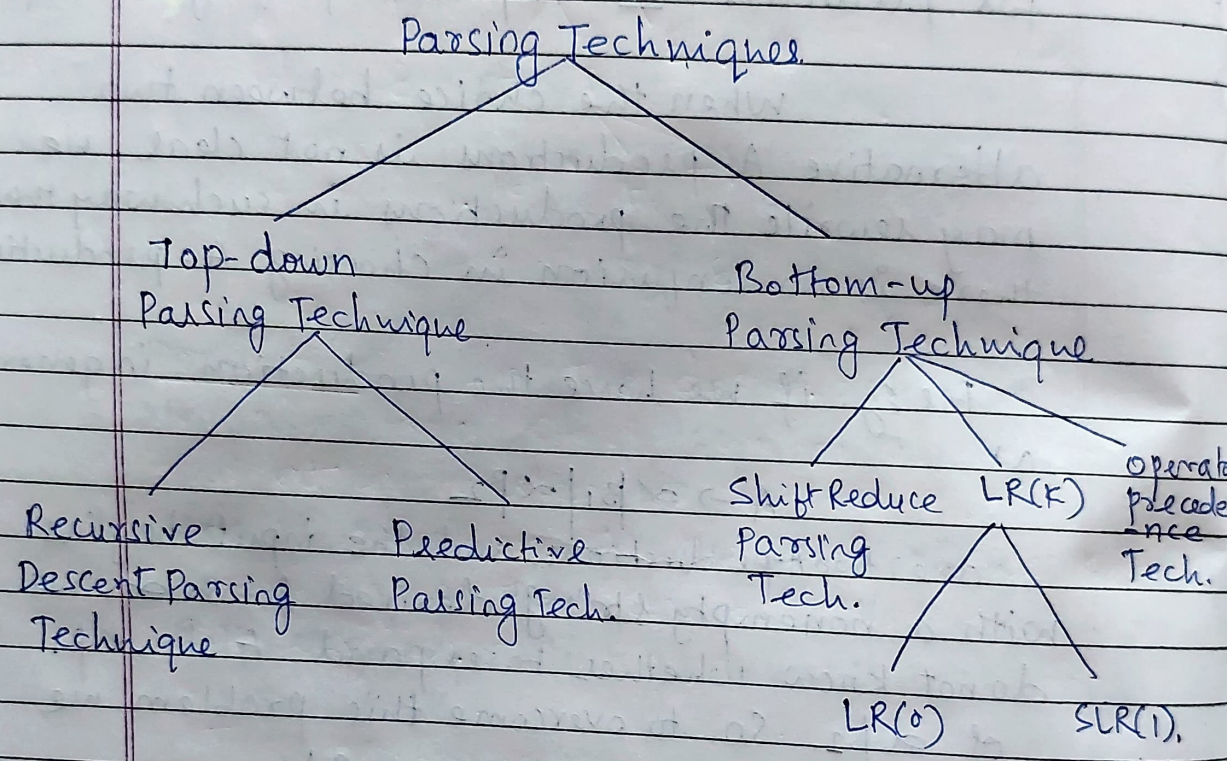
$$\left.\begin{aligned} A &\to \alpha A' \\ A' &\to \beta_1 | \beta_2 \end{aligned}\right\} \text{ Left Factored grammar.}$$

In the above modified grammar A can be expand to $\alpha A'$. Then, after seeing the input derived from α, we expand A' to β_1 or β_2.

For Eg: $S \to iEtS | iEtSeS | a$
$E \to b$

above grammar can be left factored as

$S \rightarrow iEtSS' \mid a$
$S' \rightarrow eS \mid \epsilon$
$E \rightarrow b.$



Top-down parsing Technique:

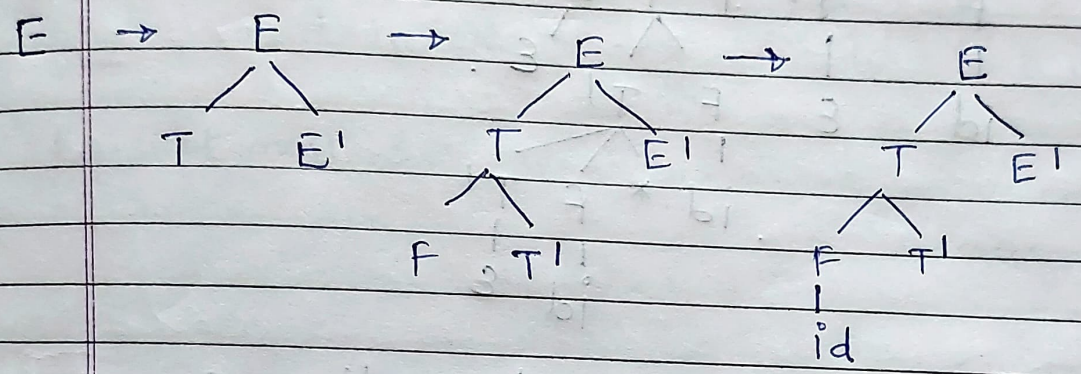
In top down parsing technique, Parse tree will be constructed from top (root) to bottom (leaves). top down parsing technique is also called as 'derivation'.

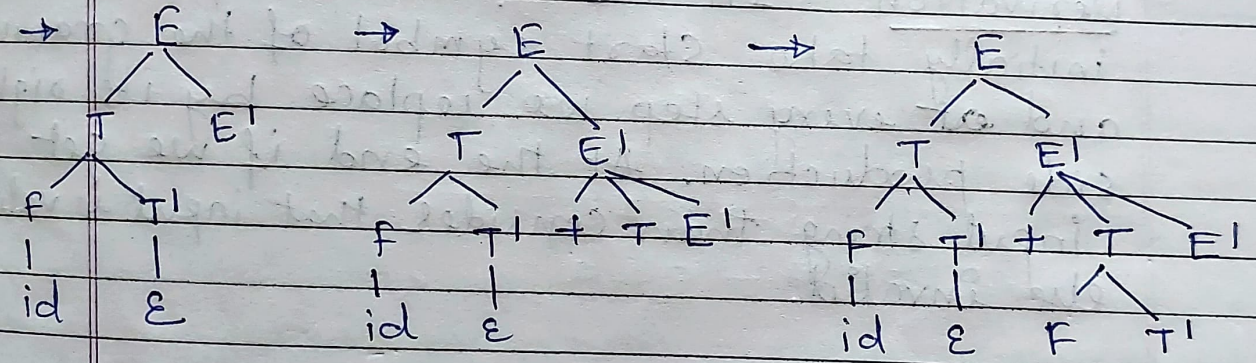
Consider a grammar

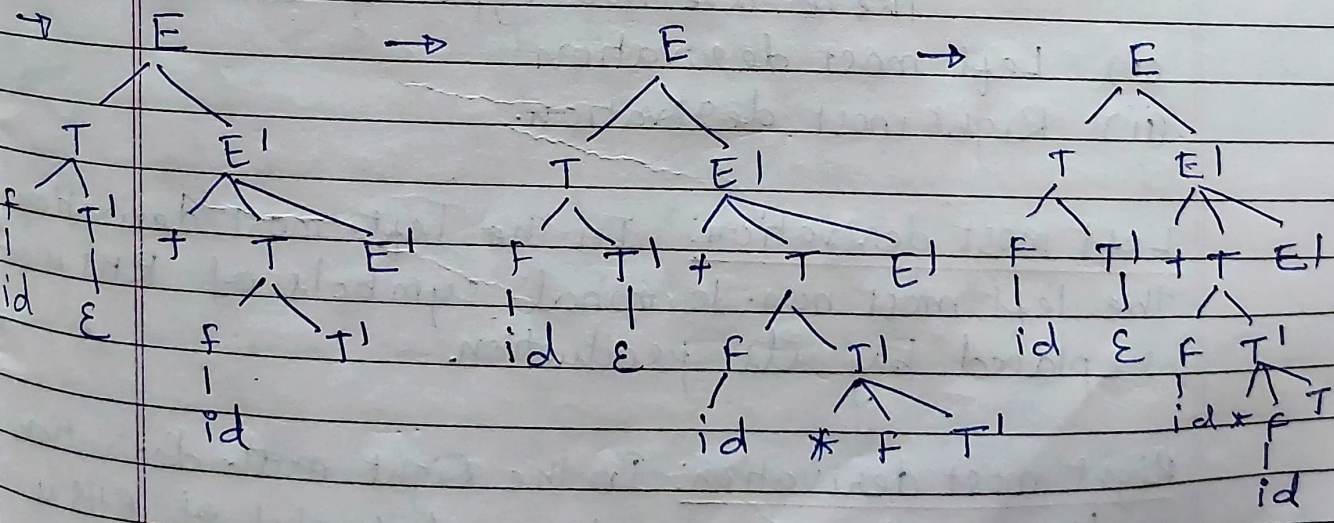
$E \rightarrow TE'$
$E' \rightarrow +TE' \mid \varepsilon$
$T \rightarrow fT'$
$T' \rightarrow *fT' \mid \varepsilon$
$F \rightarrow (E) \mid id$

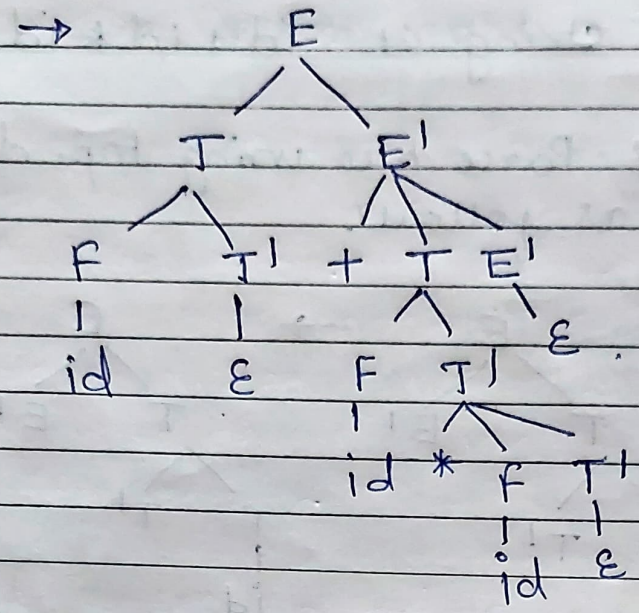
and input string as id+id*id

Construction of Parse tree using top-down parsing technique is as follows.









Derivation: In the derivation process we initially take start symbol of the grammar and at every step we replace by its right side production. At the end if we get input string then consider that input is valid else invalid.

Types of derivation:

There are two types of derivation

(i) Left most derivation
(ii) Right most derivation.

Left most derivation: In the left-most derivation the left most non-terminal symbols of RHS is replaced by its production.

Right most derivation: In the Right most-derivation the right most non-terminal symbol of RHS is replaced by its Production.

Let us consider an Eg grammar

$E \rightarrow E+T \mid T$
$T \rightarrow T*F \mid F$
$F \rightarrow (E) \mid id$

and input string is id+id*id.

Eg. Left most derivation:

$E \xrightarrow{lm} E+T$
$\xrightarrow{lm} T+T$
$\xrightarrow{lm} f+T$
$\xrightarrow{lm} id+T$
$\xrightarrow{lm} id+T*f$
$\xrightarrow{lm} id+f*f$
$\xrightarrow{lm} id+id*f$
$\xrightarrow{lm} id+id*id.$

Eg. Right most derivation:

$E \xrightarrow{rm} E+T$
$E \xrightarrow{rm} E+T*f$
$E \xrightarrow{rm} E+T*id$
$E \xrightarrow{rm} E+f*id$
$E \xrightarrow{rm} E+id*id$
$E \xrightarrow{rm} T+id*id$
$E \xrightarrow{rm} f+id*id$
$E \xrightarrow{rm} id+id*id.$

* Note: Top-down-parsing technique supports left most derivation and bottom-up parsing technique supports right most derivation.

Page No.
Date

Recursive Descent Parsing Technique:

Recursive Descent Parsing technique some-time requires backtracking process to find correct A-production.

As backtracking process increases the time and reduces the efficiency, normally recursive descent parsing technique not used practically.

Let us see the below eg. to understand recursive descent parsing technique.

Consider the grammar

$$S \rightarrow cAd$$
$$A \rightarrow ab \mid a$$

to construct a parse tree top-down for the i/p string $w = cad$, begin with a tree consisting of a single node labeled S, and the input pointer pointing to c, the first symbol of w. S has only one production, so we use it to expand S & obtain the tree of figure (a). the left most leaf, labeled c, matches the first symbol of input w, so we advance the input pointer to a, the second symbol of w, and consider the next leaf, labeled A.

Now, we expand A using the first alternative $A \rightarrow a$ to obtain the tree of fig(b). We have a match for the second input symbol, a. So, we advance the input pointer to d, the third input

symbol 'a' and compare 'd' against the next leaf, labeled b. Since b does not d, we report failure and go back to A-production too see whether there is another alternative for A that has not been tried, but that might produce a match.

In going back to A, we must reset the input pointer to position 2, the position it had we fisst came to A.

The second alternative for A produces tree of figure (c). The leaf a matches the second symbol of w and the leaf d matches the third symbol.

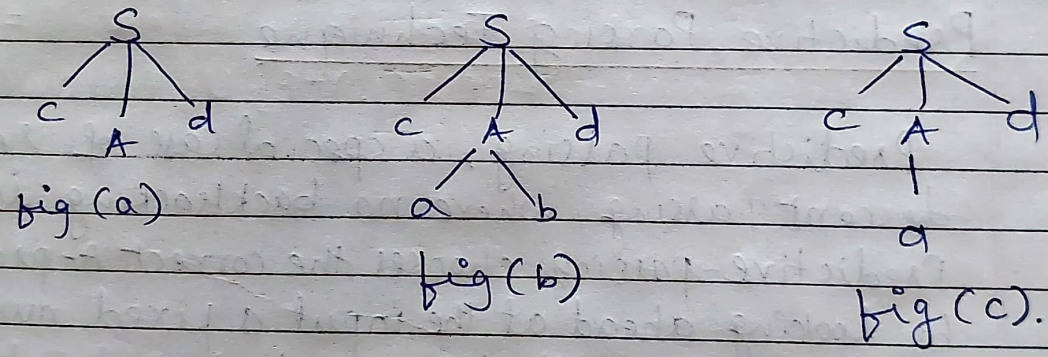


Fig: Illustrates steps in Recursive descent top-down parsing technique.

Algorithm (Recursive Descent Parsing)

```
void A(){
1. choose an A-production A→X1,X2--Xk ;
2. for (i=1 to k){
3.    if (Xi is a terminal)
4.       call procedure Xi()
5.    else if (Xi equals to current i/p symbol a)
6.       advance the i/p to next symbol;
7.    else an error.
   }
```

To allow backtracking in the above algorithm. First we cannot choose a unique A-production at line (1), so we must try each of several productions in some order. Then failure at line (7) is not ultimate failure, but suggests only that we need to return at line (1) and try another A-production.

As it requires more time to parse the input string we go to the another technique called as Predictive Parsing technique (Non-recursive parsing technique).

Predictive Parsing Technique

Predictive parsing, a special case of recursive-descent parsing, where no backtracking is required. Predictive parsing chooses the correct A-production by looking ahead at the input a fixed number of symbols. typically we may look only at one (that is, the next input symbol).

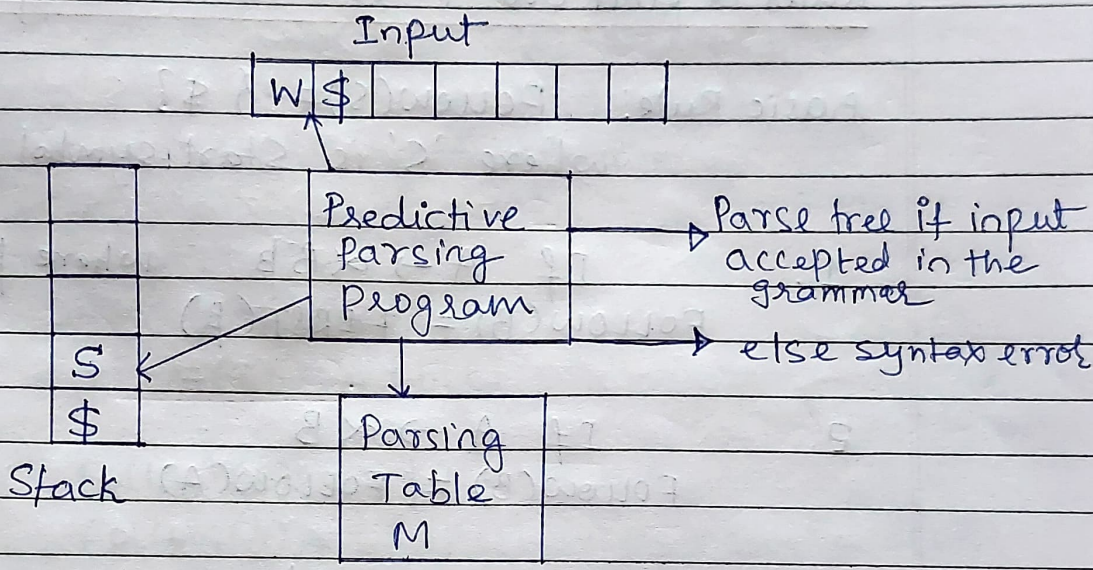
Predictive parsers, that is, recursive-descent parsers needing no backtracking can be constructed for a class of grammar called LL(1).

* LL(1) Grammar:-

The first "L" in LL(1) stands for scanning the input from Left to Right, the second 'L' for producing left most derivation, and the '1' for using one input symbol of lookahead at

each step to make parsing action decisions.

Model of Predictive Parser of LL(1) grammar:-



In above Predictive parser model, we will be given Input 'w', a grammar G and Predictive parsing program. Now we need to construct predictive parsing table M.

To construct predictive parsing table M, we need to find out FIRST set and FOLLOW set of the given grammar.

Rules to find out FIRST set of the grammar.

Basic Rule: FIRST(terminal) = { terminal }

1. If $A \to a\alpha$
   FIRST(A) = {a}

2. If $A \to B\alpha$ where $B \neq \varepsilon$
   FIRST(A) = FIRST(B)

3. If $A \to B\alpha$ where $B = \varepsilon$

$FIRST(A) = \{FIRST(B) - \varepsilon\} \cup \{FIRST(\alpha)\}$

Rules to find out FOLLOW set of the grammar

Basic Rule: $FOLLOW(S) = \{\$\}$

where 'S' is start symbol of the grammar

1. If $A \to \alpha B \beta$ where $\beta \neq \varepsilon$

$FOLLOW(B) = FIRST(\beta)$

2. If $A \to \alpha B$

$FOLLOW(B) = FOLLOW(A)$

3. If $A \to \alpha B \beta$ where $\beta = \varepsilon$

$FOLLOW(B) = \{FIRST(\beta) - \varepsilon\} \cup \{FOLLOW(A)\}$

Example:

1. $S \to AB$

   $A \to a$

   $B \to b$

Find out FIRST set and FOLLOW set of the grammar.

$FIRST(S) = FIRST(A) = \{a\}$

$FIRST(B) = \{b\}$

$FOLLOW(S) = \{\$\}$

$FOLLOW(A) = FIRST(B) = \{b\}$

$FOLLOW(B) = FOLLOW(S) = \{\$\}$

2. $S \to AB$
$A \to a/\varepsilon$
$B \to b/\varepsilon$

$FIRST(S) = \{FIRST(A) - \varepsilon\} \cup \{FIRST(B)\}$
$= \{(a, \varepsilon) - \varepsilon\} \cup \{b\}$
$= \{a\} \cup \{b\}$
$= \{a, b\}$

$FIRST(A) = \{a, \varepsilon\}$
$FIRST(B) = \{b, \varepsilon\}$

$FOLLOW(S) = \{\$\}$
$FOLLOW(A) = \{FIRST(B) - \varepsilon\} \cup \{FOLLOW(S)\}$
$= \{(b, \varepsilon) - \varepsilon\} \cup \{\$\}$
$= \{b\} \cup \{\$\}$
$= \{b, \$\}$

3. $S \to iEtS/iEtSeS/b$
$E \to a$

$FIRST(S) = \{i, b\}$
$FIRST(E) = \{a\}$

$FOLLOW(S) = \{\$, e\}$
$FOLLOW(E) = \{t\}$

4. $E \to TE'$
$E' \to +TE'/\varepsilon$
$T \to FT'$
$T' \to *FT'/\varepsilon$
$F \to (E)/id$

FIRST(E) = FIRST(T) = FIRST(F) = { (, id }
FIRST(E') = { +, ε }
FIRST(T') = { *, ε }

FOLLOW(E) = { $,) }
FOLLOW(E') = FOLLOW(E) = { $,) }
FOLLOW(T) = { FIRST(E') - ε } ∪ { FOLLOW(E) }
= { (+, ε) - ε } ∪ { ($,) }
= { + } ∪ { $,) }
= { +, $,) }
FOLLOW(T') = FOLLOW(T) = { +, $,) }
FOLLOW(F) = { FIRST(T') - ε } ∪ { FOLLOW(T) }
= { (*, ε) - ε } ∪ { +, $,) }
= { * } ∪ { +, $,) }
= { +, *, $,) }

Once we find out FIRST set and FOLLOW set of the grammar, we can construct Predictive Parsing table M.

Let us consider below grammar

1. S → AB
   A → a
   B → b

	FIRST	FOLLOW
S	{ a }	{ $ }
A	{ a }	{ b }
B	{ b }	{ $ }

M	a	b	$
S	S → AB		
A	A → a		
B		B → b	

As table does not contain multiple entries, we can say grammar is LL(1).

* [[Question to be asked in the exam:
check whether the given grammar is in LL(1) or not]]

** Note:- 1. If table contains multiple entries then, we say that the given grammar is not in LL(1) and hence it is ambigous grammar. If the grammar is ambigous then it cannot be used to parse any input string.
2. If table does not contain any multiple entries as in the above example, then we say that the given grammar is in LL(1) and hence it is unambigous grammar and can be used to parse any valid input string.

Example - 2.

$E \to TE'$
$E' \to +TE' | \varepsilon$
$T \to FT'$
$T' \to *FT' | \varepsilon$
$F \to (E) | id$

	FIRST	FOLLOW
E	{(, id}	{$,)}
E'	{+, ε}	{$,)}
T	{(, id}	{+, $,)}
T'	{*, ε}	{+, $,)}
F	{(, id}	{+, *, $,)}

M	+	*	(	)	id	$
E			$E \to TE'$		$E \to TE'$	
E'	$+TE'$			$E' \to \epsilon$		$E' \to \epsilon$
T			$T \to FT'$		$T \to FT'$	
T'	$T' \to \varepsilon$	$T' \to *FT'$		$T' \to \epsilon$		$T' \to \epsilon$
F			$F \to (E)$	..	$F \to id$	

As above table does not contain multiple entries, hence the grammar is in LL(1).

Example-3

$S \to iEtSS' | a$
$S' \to eS | \epsilon$
$E \to b$

	FIRST	FOLLOW
S	{i}	{$, e}
S'	{e, ε}	{$, e}
E	{b}	{t}

M	i	t	e	b	$
S	$S \to iEtSS'$				
S'			$S' \to eS$ $S' \to \epsilon$		$S' \to \epsilon$
E				$E \to b$	

As above table contains multiple entries at M[S', e], the grammar is not in LL(1) and hence it is ambigous grammar

* Note:- while constructing Predictive Parsing table, if FIRST set of any non-terminal contains 'ε' in it then only we look for FOLLOW set of that Particular non-terminal, to fill the table.

If the table does not contain any multiple entries then grammar is in LL(1) and hence it is unambigous grammar and consequently it can be used to parse any valid input string.

Let us consider example-2 to parse the input string id+id*id$.

As predictive parsing table for example-2 is already constructed, we directly parse the input string as follows.

Matched	Stack	Input	Action
	E$	id+id*id$	
	TE'$	id+id*id$	output E→TE'
	FT'E'$	id+id*id$	output T→FT'
	idT'E'$	id+id*id$	output f→id
id	T'E'$	+id*id$	match id
id	E'$	+id*id$	output T'→ε
id	+TE'$	+id*id$	output E'→+TE'
id+	TE'$	id*id$	match +
id+	FT'E'$	id*id$	output T→FT'
id+	idT'E'$	id*id$	output F→id
id+id	T'E'$	*id$	match id
id+id	*FT'E'$	*id$	output T'→*FT'
id+id*	FT'E'$	id$	match *
id+id*	idT'E'$	id$	output F→id
id+id*id	T'E'$	$	match id
id+id*id	E'$	$	output T'→ε
id+id*id	$	$	output E'→ε

* Predictive parsing algorithm

Input: A string 'w', a parsing table M and a grammar G.

output: If 'w' is in L(G), a leftmost derivation of w; otherwise an error indication.

Method: Initially, the parser is in a configuration with w$ in the input buffer and the start symbol S of G on top of the stack, above $.

let a be the first symbol of w;
let X be the top of the stack symbol;
while (X ≠ $){ /* stack not empty */
if (X=a) pop the stack and let a be the next symbol of w;
else if (X is a terminal) error();
else if (M[X,a] is an error entry) error();
else if (M[X,a] = X → Y1, Y2, ... Yk;){
output the production X → Y1, Y2 .... Yk;
pop the stack
push Yk, Yk-1 --- Y1 onto the stack, with Y1 on top;
}
};

Bottom-up parsing Technique:

Bottom up parse corresponds to the construction of a parse tree for an input string beginning at the leaves (bottom) and working up towards the root (the top).

For Eg:

Consider a grammar

E → E+T | T
T → T*F | F
F → (E) | id

and Input string id * id.

. Bottom up parse tree generated for above input using above grammar is as follows.

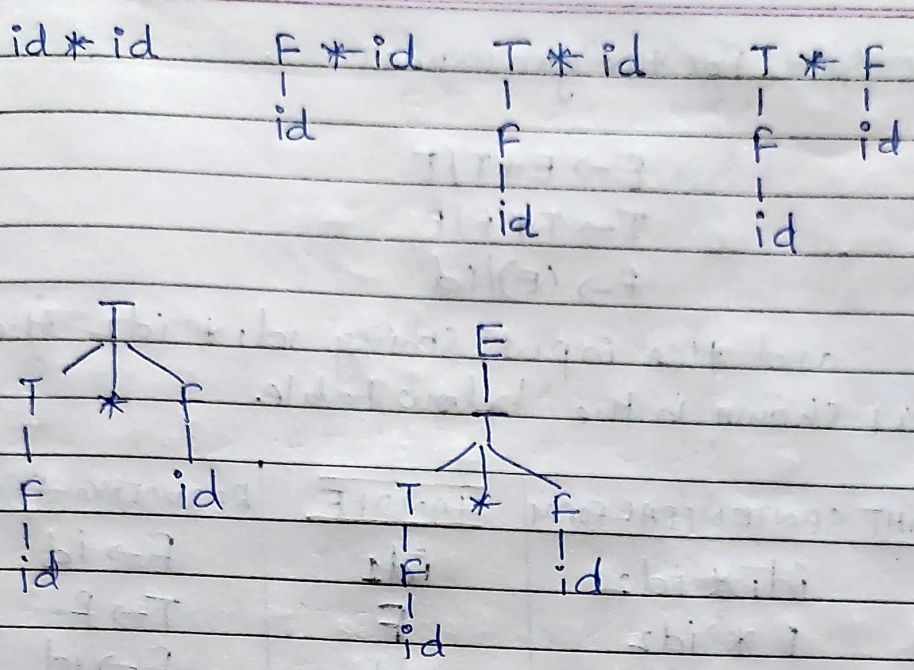


Fig: illustrates A bottom up parse tree for id*id

Bottom up parsing is also known as process of "reduction" a string 'w' to the start symbol of the grammar. At each reduction step, a specific substring matching the body of a production is replaced by the non-terminal at the left side of the production.

For eg: id*id
F*id
T*id
T*F
T
E

Handle: Handle is a substring of the input string, when it matches RHS of the production, whose reduction will gives us start symbol of the grammar.

For Eg: For the grammar

$E \rightarrow E+T \mid T$
$T \rightarrow T*f \mid f$
$f \rightarrow (E) \mid id$

and the input string $id_1 * id_2$ the handle is as shown in the below table.

RIGHT SENTENTIAL FORM	HANDLE	REDUCING PRODUCTION
$id_1 * id_2$	id_1	$f \rightarrow id$
$f * id_2$	F	$T \rightarrow f$
$T * id_2$	id_2	$f \rightarrow id$
$T * f$	$T * f$	$T \rightarrow T * f$
T	T	$E \rightarrow T$
E		

Shift-Reduce Parsing :-

Shift-Reduce Parsing is a form of bottom-up parsing in which a stack holds grammar symbols and an input buffer holds input string to be parsed.

We use $ to mark the bottom of the stack and also to represent end of the input string.

Initially, the stack is empty, and the string w is on the input, as follows

STACK	Input
$	w$

Final configuration of stack and input after

Page No.
Date

Successful completion of Parsing the input string is,

STACK	INPUT
$S	$

where 'S' is start symbol of the grammar.

Shift-Reduce parsing technique uses 4-actions
(i) Shift
(ii) Reduce
(iii) Accept
(iv) Error

(i) Shift: Shift the i/p symbol onto the top of the Stack.
(ii) Reduce: When top of the stack matches RHS of the production, then it can be reduced to left side of the production.
(iii) Accept: If input is successfully parsed, then it is an accept action.
(iv) Error: else an error and call an error recovery routine.

For Eg: Consider a grammar

1. S → AB
A → a
B → b

and input string ab

Stack	Input	Action
$	ab$	Shift a
$a	b$	Reduce A→a
$A	b$	Shift b
$Ab	$	Reduce B→b
$AB	$	Reduce S→AB
$S	$	Accept

total actions used to parse the input is '6'.

3. Construct Finite automata.

Note: Step-2 & 3 must be performed simultaneo-usly.

Example-1:

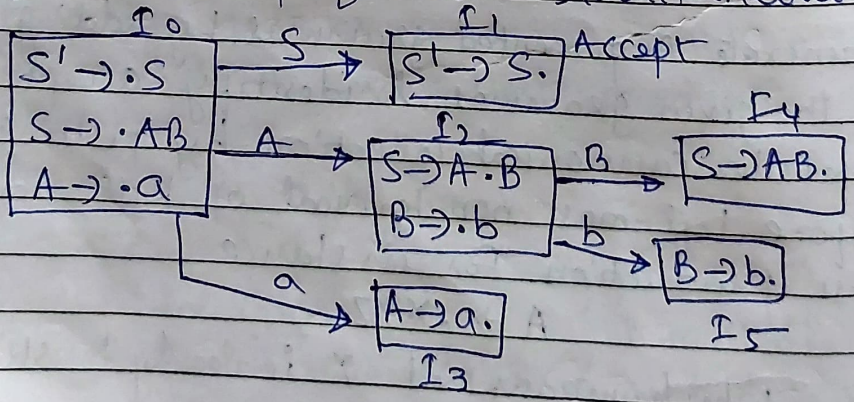
Consider grammar

$S \to AB$
$A \to a$
$B \to b$

Step: 1. Generate augumented grammar by adding $S' \to S$ to the given grammar

$S' \to S$
$S \to AB$
$A \to a$
$B \to b$
} Augumented grammar

Step: 2 & 3 will be performed simultaneously.



Now we can construct LR-passing table as follows.

	ACTION			GOTO		
	a	b	$	S	A	B
I_0	S_3			1	2	
I_1			Accept			
I_2		S_5				4
I_3	r_2	r_2	r_2			
I_4	r_1	r_1	r_1			
I_5	r_3	r_3	r_3			

To enter reduce entries, we need to serialize the given grammar like as below

1 $S \rightarrow AB$

2 $A \rightarrow a$

3 $B \rightarrow b$

and look where above productions are reduced in the finite automata, on that particular state need to enter 'r' along with production number as suffix with 'r' on all the input symbol (ACTION).

Since the technique is LR(0), it does not decide where exactly input is reduced, so needs to enter 'r' on all the input symbol in ACTION part of the table.

As the above parsing table does not contain multiple entries, we can say grammar in LR(0).

If the table does not contain any multiple entries then the grammar is in LR(0) and hence the grammar is unambigous grammar and consequently it can be used for parsing any valid input string.

Let us consider example-1 to parse the input string "ab".

Stack	Symbol	Input	Action
0	$	ab$	Shift 3
0,3	$a	b$	Reduce A→a
0,2	$A	b$	Shift 5
0,2,5	$Ab	$	Reduce B→b
0,2,4	$AB	$	Reduce S→AB
0,1	$S	$	Accept

Algorithm for LR(0):— (LR-parsing algorithm)

Input: An input string 'w' and LR-parsing table 'M' with ACTION and GOTO function, and a grammar G.

Output: If 'w' is in L(G), the reduction steps of bottom-up parse for w; otherwise an error indication.

Method:- Initially, the parser has 0 on its stack, where '0' is the initial state, and w$ in the input buffer.

```
let a be the first symbol of w$;
while (1) { /* repeat forever */
    let s be the state on top of the stack;
  if (ACTION[s,a] = shift t) {
        push 't' onto the stack;
    let a be the next input symbol;
  } else if (ACTION[s,a] = reduce A→β){
        pop |β| symbols off the stack;
    let state 't' now be on top of the stack;
```

push GOTO[t,A] onto the stack;
} else if (ACTION[s,a] = Accept) break;
else call error recovery techniques;
}

Drawbacks of LR(0) parsers:

1. LR(0) accepts only small class of LR(0) grammar because of SR & RR - conflicts.
2. The fundamental limitation of LR(0) is that no look ahead symbols are used.

Note: In LR(0), whenever we get a states lik

$A \to \alpha . a\beta$
$B \to \gamma .$

we say state contains both shift and reduce action. So, we say this situation as SR-conflict and hence can conclude the grammar is not in LR(0).

Similarly if we get a state like

$A \to \alpha .$
$B \to \gamma .$

We say this situation as RR-conflict and hence can conclude the grammar is not in LR(0)

Example for Practice:

1. $E \rightarrow E+T$
   $E \rightarrow T$
   $T \rightarrow T*F$
   $T \rightarrow f$
   $F \rightarrow (E)$
   $F \rightarrow id$

Check the given grammar is in LR(0) or not

2. $E \rightarrow T+E$
   $E \rightarrow T$
   $T \rightarrow i$

check the given grammar is LR(0) or not.

To overcome the problem of LR(0), we goto other technique called as SLR(1).

SLR(1)

'S' stands for simple, 'L' stands for left to right and 'R' stands for right most derivation, '1' indicates one symbol lookahead at every reduction step.

SLR(1) is same as that of LR(1) except entering reduce entries in the parsing table.

Also whenever we get SR-conflict and RR-conflict kind of states, in SLR(1) we cannot say directly that the state contains SR-conflict and RR conflict

instead we need to find out following.

* If SLR(1) contains a state like

$A \to \alpha . a\beta$
$B \to \gamma .$

we cannot directly say, the state is SR-conflict instead we need to find

FOLLOW(B)

if FOLLOW(B) = {a} (symbol after '.')
then we say that state contains SR conflict and hence grammar is not in SLR(1)

* If SLR(1) contains a state like

$A \to \alpha .$
$B \to \gamma .$

we cannot directly say, the state is RR-conflict instead we need to find

FOLLOW(A) & FOLLOW(B)

if FOLLOW(A) ∩ FOLLOW(B) ≠ {ϕ}
then we say that state contains RR-conflict and hence grammar is not in SLR(1).